

Index

SL NO	ASSIGNMENT NAME	DATE OF EXPERIMENT	PAGE NO
01	Write down a simple server client programme using TCP.		1
02	Write down a simple server client programme using TCP.		3
03	Write down a simple server client programme using UDP.		4
04	Write down a simple server client programme using UDP.		6
05	Write down a programme which send current date and time to client from server.		7
06	Write down a programme which send current date and time to client from server		9
07	Write down a programme which square a given number requested from client		11
08	Write down a simple programme which square a given number requested from client		13
09	Implement Character Count on Given Data.		15
10	Implement a Program to Decode the Stuffed Data Using C.		17
11	Implement Bit Stuffing on Given Data.		18
12	Implement CRC Code Generation for Given Data.		20
13			
14			
15			
16			
17			
18			
19			
20			

Teacher' s

Assignment NO:01**Assignment Name:** Write down a simple server client programme using TCP.**Source Code of server side:**

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include<unistd.h>
#include<sys/types.h>
#include<sys/socket.h>
#include<sys/un.h>
#define MAXSIZE 100
#define PATH "/tmp/mysock"

int main()
{
    int sd,bv,lv,av,r;
    char buffer[MAXSIZE];
    socklen_t l;
    struct sockaddr_un myaddr;
    unlink(PATH);
    sd=socket(AF_UNIX,SOCK_STREAM,0);
    if(sd== -1)
    {
        perror("socket_err");
        exit(1);
    }
    myaddr.sun_family=AF_UNIX;
    strncpy(myaddr.sun_path,PATH,sizeof(PATH));
    l=sizeof(myaddr);
    bv=bind(sd,(struct sockaddr*)&myaddr,l);
    if(bv== -1)
    {
        perror("bind_err");
        exit(2);
    }
    lv=listen(sd,10);
    if(lv== -1)
    {
        perror("listen_err");
        exit(3);
    }
    while(1)
```



```
{
    printf("waiting for client request\n");
    av=accept(sd,(struct sockaddr*)&myaddr,&l);
    if(av==-1)
    {
        perror("accept_err");
        exit(4);
    }
    memset(buffer,'\0',MAXSIZE);
    r=read(av,buffer,MAXSIZE);
    write(av,buffer,r);
    close(av);
}
return 0;
}
```

Output:

user@ubuntu:~/Desktop\$ gcc server.c -o server

user@ubuntu:~/Desktop\$./server

waiting for client request

Assignment NO:02**Assignment Name:** Write down a simple server client programme using TCP.**Source Code of client side:**

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include<unistd.h>
#include<sys/types.h>
#include<sys/socket.h>
#include<sys/un.h>

#define MAXSIZE 100
#define PATH "/tmp/mysock"

int main()
{
    int sd, cv, r;
    char buffer[MAXSIZE];
    struct sockaddr_un myaddr;
    sd = socket(AF_UNIX, SOCK_STREAM, 0);
    if(sd == -1)
    {
        perror("socket_err");
        exit(1);
    }
    myaddr.sun_family = AF_UNIX;
    strncpy(myaddr.sun_path, PATH, sizeof(PATH));
    cv = connect(sd, (struct sockaddr *)&myaddr, sizeof(myaddr));
    if(cv == -1)
    {
        perror("connect_err");
        exit(2);
    }
    printf("Enter a message : ");
    fgets(buffer, MAXSIZE, stdin);
    write(sd, buffer, strlen(buffer));
    memset(buffer, '\0', MAXSIZE);
    r = read(sd, buffer, MAXSIZE);
    printf("Received from server : %s\n", buffer);
    close(sd);
    return 0;
}
```

Output:

```
user@ubuntu:~/Desktop$ gcc client.c -o client
```

```
user@ubuntu:~/Desktop$ ./client
```

```
Enter a message : Hello
```

```
Received from server : Hello
```

Assignment NO:03

Assignment Name: Write down a simple server client programme using UDP.

Source Code of server side:

```
#include<stdio.h>
#include<unistd.h>
#include<string.h>
#include<sys/types.h>
#include<sys/socket.h>
#include<stdlib.h>
#include<sys/un.h>

#define MAXSIZE 100
#define PATH "/tmp/mysock"

int main()
{
    int sd,bv,r,w;
    char buffer[1000];
    socklen_t sl,cl;
    struct sockaddr_un server_addr,client_addr;
    unlink(PATH);
    sd=socket(AF_UNIX,SOCK_DGRAM,0);
    if(sd==-1)
    {
        perror("socket_error");
        exit(1);
    }
    client_addr.sun_family=AF_UNIX;
    cl=sizeof(server_addr);
    bv=bind(sd,(struct sockaddr*)&client_addr,cl);
    if(bv==-1)
    {
        perror("bind_error");
        exit(2);
    }
    printf("waiting for client request----\n");
    memset(buffer,'\0',100);
```



```
    r=read(0,buffer,MAXSIZE);
    server_addr.sun_family=AF_UNIX;
    strncpy(server_addr.sun_path,PATH,sizeof(PATH));
    sl=sizeof(server_addr);
    w=sendto(sd,buffer,r,0,(struct sockaddr*)&server_addr,sl);
    memset(buffer,'\0',MAXSIZE);
    recvfrom(sd,buffer,MAXSIZE,0,(struct sockaddr*)&server_addr,&sl);
    write(1,buffer,MAXSIZE);
    return 0;
}
```

Output:

```
user@ubuntu:~/Desktop$ gcc serverUDP.c -o serverUDP
```

```
user@ubuntu:~/Desktop$ ./serverUDP
```

```
waiting for client request----
```

Assignment NO:04**Assignment Name:** Write down a simple server client programme using UDP.**Source Code of client side:**

```

#include<stdio.h>
#include<unistd.h>
#include<string.h>
#include<sys/types.h>
#include<sys/socket.h>
#include<stdlib.h>
#include<sys/un.h>

#define MAXSIZE 100
#define PATH "/tmp/mysock"

int main()
{
    int sd,r,w;
    char buffer[MAXSIZE];
    socklen_t sl,cl;
    struct sockaddr_un server_addr,client_addr;
    unlink(PATH);
    sd=socket(AF_UNIX,SOCK_DGRAM,0);
    if(sd== -1)
    {
        perror("socket_error");
        exit(1);
    }
    server_addr.sun_family=AF_UNIX;
    strncpy(server_addr.sun_path,PATH,sizeof(PATH));
    sl=sizeof(server_addr);
    printf("Enter a message:");
    memset(buffer,'\0',MAXSIZE);
    r=read(0,buffer,MAXSIZE);
    w=sendto(sd,buffer,r,0,(struct sockaddr*)&server_addr,sl);
    memset(buffer,'\0',MAXSIZE);
    recvfrom(sd,buffer,MAXSIZE,0,(struct sockaddr*)&client_addr,&cl);
    write(1,buffer,MAXSIZE);
    close(sd);
    return 0;
}

```

Output:


```
user@ubuntu:~$ gcc program.c -o program
```

```
user@ubuntu:~$ ./program
```

```
Enter string
```

```
hello
```

Assignment NO:05

Assignment Name: Write down a programme which send current date and time to client from server.

Source Code of server side:

```
#include<stdio.h>
#include<string.h>
#include<unistd.h>
#include<stdlib.h>
#include<sys/types.h>
#include<sys/socket.h>
#include<sys/stat.h>
#include<fcntl.h>
#include<sys/un.h>

#define MAXSIZE 100
#define PATH "/tmp/mysock"

int main()
{
    int sd,bv,lv,av,r,dat;
    char buffer[100];
    socklen_t l;
    struct sockaddr_un myaddr;
    unlink(PATH);
    sd=socket(AF_UNIX,SOCK_STREAM,0);
    if(sd== -1)
    {
        perror("socket_err");
        exit(1);
    }
    myaddr.sun_family=AF_UNIX;
    strncpy(myaddr.sun_path,PATH,sizeof(PATH));
    l=sizeof(myaddr);
    bv=bind(sd,(struct sockaddr*)&myaddr,l);
    if(bv== -1)
    {
        perror("bind_err");
        exit(2);
    }
    lv=listen(sd,10);
    if(lv== -1)
    {
```

```
    perror("listen_err");
    exit(3);
}
while(1)
{
    printf("waiting for client request-----\n");
    av=accept(sd,(struct sockaddr*)&myaddr,&l);
    if(av== -1)
    {
        perror("accept_err");
        exit(4);
    }
    memset(buffer,'\0',MAXSIZE);
    r=read(av,buffer,MAXSIZE);
    system("date>abc.txt");
    dat=open("abc.txt",O_RDONLY);
    r=read(dat,buffer,MAXSIZE);
    write(av,buffer,r);
    close(av);
}
return 0;
}
```

Output:

```
user@ubuntu:~/Desktop$ gcc server 1.c -o server1
user@ubuntu:~/Desktop$ ./server1
waiting for client request-----
```


Assignment NO:06

Assignment Name: Write down a programme which send current date and time to client from server

Source Code of client side:

```
#include<stdio.h>
#include<string.h>
#include<stdlib.h>
#include<sys/types.h>
#include<sys/un.h>
#include<sys/socket.h>
#include<unistd.h>
#include<sys/stat.h>
#include<fcntl.h>

#define MAXSIZE 100
#define PATH "/tmp/mysock"

int main()
{
    int sd,cv,r;
    char buffer[100];
    socklen_t l;
    struct sockaddr_un myaddr;
    sd=socket(AF_UNIX,SOCK_STREAM,0);
    if(sd==-1)
    {
        perror("socket_err");
        exit(1);
    }
    myaddr.sun_family=AF_UNIX;
    strncpy(myaddr.sun_path,PATH,sizeof(PATH));
    l=sizeof(myaddr);
    cv=connect(sd,(struct sockaddr*)&myaddr,l);
    if(cv==-1)
    {
        perror("connect_err");
        exit(2);
    }
    printf("Enter a number\n");
    memset(buffer,'\0',MAXSIZE);
    r=read(0,buffer,MAXSIZE);
    write(sd,buffer,r);
```

```
printf("Echo date\n");  
memset(buffer,'\0',MAXSIZE);  
r=read(sd,buffer,MAXSIZE);  
write(1,buffer,r);  
return 0;  
}
```

Output:

```
user@ubuntu:~/Desktop$ gcc client1.c -o client1  
user@ubuntu:~/Desktop$ ./client1  
Enter a number  
1  
Echo date  
Thu Jun 4 05:54:01 PM IST 2026  
user@ubuntu:~/Desktop$
```

Assignment NO:07

Assignment Name: Write down a programme which square a given number requested from client

Source Code of server side:

```
#include<stdio.h>
#include<string.h>
#include<stdlib.h>
#include<sys/types.h>
#include<sys/un.h>
#include<sys/socket.h>
#include<unistd.h>

#define MAXSIZE 100
#define PATH"/tmp/mysock"

int main()
{
    int sd,bv,lv,av,r,n;
    char buffer[1000];
    socklen_t l;
    struct sockaddr_un myaddr;
    unlink(PATH);
    sd=socket(AF_UNIX,SOCK_STREAM,0);
    if(sd== -1)
    {
        perror("socket_err");
        exit(1);
    }
    myaddr.sun_family=AF_UNIX;
    strncpy(myaddr.sun_path,PATH,sizeof(PATH));
    l=sizeof(myaddr);
    bv=bind(sd,(struct sockaddr*)&myaddr,l);
    if(bv== -1)
    {
        perror("bind_err");
        exit(2);
    }
    lv=listen(sd,10);
    if(lv== -1)
    {
        perror("listen_err");
        exit(3);
    }
}
```



```
    }  
    while(1)  
    {  
        printf("waiting for client request\n");  
        av=accept(sd,(struct sockaddr*)&myaddr,&l);  
        memset(buffer,'\0',MAXSIZE);  
        r=read(av,buffer,MAXSIZE);  
        n=atoi(buffer);  
        sprintf(buffer,"%d",n*n);  
        write(av,buffer,10);  
        close(av);  
    }  
    return 0;  
}
```

Output:

```
user@ubuntu:~/Desktop$ gcc server7.c -o server7
```

```
user@ubuntu:~/Desktop$ ./server7
```

```
waiting for client request
```

Assignment NO:08

Assignment Name: Write down a simple programme which square a given number requested from client

Source Code of Client:

```
#include<stdio.h>
#include<string.h>
#include<stdlib.h>
#include<sys/types.h>
#include<sys/socket.h>
#include<sys/un.h>
#include<unistd.h>

#define MAXSIZE 100
#define PATH "/tmp/mysock"

int main()
{
    int sd,cv,r,no;
    char buffer[1000];
    socklen_t l;
    struct sockaddr_un myaddr;
    sd=socket(AF_UNIX,SOCK_STREAM,0);
    if(sd==-1)
    {
        perror("socket_err");
        exit(1);
    }
    myaddr.sun_family=AF_UNIX;
    strncpy(myaddr.sun_path,PATH,sizeof(PATH));
    l=sizeof(myaddr);
    cv=connect(sd,(struct sockaddr*)&myaddr,l);
    if(cv==-1)
    {
        perror("connect_err");
        exit(2);
    }
    printf("Enter a number\n");
    memset(buffer,'\0',100);
    r=read(0,buffer,MAXSIZE);
    write(sd,buffer,r);
    printf("Square of the number\n");
    memset(buffer,'\0',100);
```



```
    r=read(sd,buffer,MAXSIZE);  
    no=atoi(buffer);  
    printf("%d\n",no);  
    return 0;  
}
```

Output:

```
user@ubuntu:~$ gcc client7.c -o client7  
user@ubuntu:~$ ./client7  
Enter a number  
4  
Square of the number  
16
```

Assignment NO:

Assignment Name: Implement Character Count on Given Data.

Source Code:

```
#include<stdio.h>
#include<string.h>

int main()
{
    int i,j,k,l,count=0,n;
    char s[100],cs[50];
    printf("\n ENTER THE BIT STRING:");
    gets(s);
    n=strlen(s);
    printf("\n THE STRING IS\n");
    for(i=0;i<n;)
    {
        if(s[i]==s[i+1])
        {
            count=2;
            i++;
            while(s[i]==s[i+1])
            {
                i++;
                count++;
            }
            if(count>=5)
            {
                printf("$");
                if(count<10)
                    printf("0");
                printf("%d%c",count,s[i]);
                i++;
            }
            else
            {
                for(j=0;j<count;j++)
                    printf("%c",s[i]);
                i++;
            }
        }
        else
        {
            {
```

```
        printf("%c",s[i]);
        i++;
    }
}
return 0;
}
```

Output:

```
ENTER THE BIT STRING:123AAAAAAAAAAATYKKKKPPPPP
THE STRING IS
123$10ATYKKKK$05P
```

Process exited after 2.705 seconds with return value 0
Press any key to continue...

Assignment NO:**Assignment Name:** Implement a Program to Decode the Stuffed Data Using C.**Source Code:**

```
#include<stdio.h>
#include<string.h>
#include<stdlib.h>
int main()
{
    int i,j,k,l,n,count;
    char s[100],cs[50];
    printf("\nENTER THE STUFFED STRING:");
    gets(s);
    n=strlen(s);
    printf("\nTHE STRING IS\n");
    for(i=0;i<n;)
    {
        if(s[i]!='$')
        {
            i++;
            count=(s[i]-'0')*10+(s[i+1]-'0');
            if(count<5)
            {
                printf("INVALID MESSAGE");
                exit(1);
            }
            while(count>0)
            {
                printf("%c",s[i+2]);
                count--;
            }
            i=i+3;
        }
        else
        {
            printf("%c",s[i]);
            i++;
        }
    }
    return 0;
}
```

Output:

ENTER THE STUFFED STRING:123\$10ATY\$06K
 THE STRING IS
 123AAAAAAAAAAATYKKKKKK

 Process exited after 28.82 seconds with return value 0
 Press any key to continue ...

Assignment NO:

Assignment Name: Implement Bit Stuffing on Given Data.

Source Code:

```
#include<stdio.h>
#include<conio.h>
#include<math.h>

int main()
{
    int b[100],b1[100],l,k,n=0,i,j,z,i1,s[20],f[8]={0,1,1,1,1,1,0},j1;
    static int a[100];
    char ch='y',bs[50];
    do
    {
        i1=z=n=0;
        printf("\n Enter the bit string(space for each byte)");
        gets(bs);

        for(i=0;bs[i]!='\0';i++)
            if(bs[i]!=' ')
                b[n++]=bs[i]-'0';

        for(i=0;i<n;i++)
        {
            if(b[i]==1)
            {
                i1++;
                if(i1==5)
                {
                    s[z++]=i+1;
                    i1=0;
                }
            }
            else
                i1=0;
        }
    }
```

```
j1=j=0;

for(i=0;i<z;i++)
{
    while(j<s[i])
        b1[j1++]=b[j++];

    b1[j1++]=0;
}

while(j<n+z)
    b1[j1++]=b[j++];

l=n/8;

for(i=0;l>0;i++)
{
    a[i]=l%2;
    l=l/2;
}

printf("\nAfter stuffing:");

for(j=7;j>=0;j--)
    printf("%d",a[j]);

printf("");

for(k=0;k<8;k++)
    printf("%d",f[k]);

printf("");

for(k=0;k<j1;k++)
    printf("%d",b1[k]);

printf("");

for(k=0;k<8;k++)
    printf("%d",f[k]);

printf("\n\nDo u want to continue?");
```



```
        ch=getch();
    }
    while(ch=='y' || ch=='Y');

    return 0;
}
```

Output:

Enter the bit string(space for each byte)1111111011111100011110

After stuffing :000000110111110 1111011101111010001111008134246081268000111110

Do u want to continue?

Assignment NO:

Assignment Name: Implement CRC Code Generation for Given Data.

Source Code:

```
#include<stdio.h>
#include<math.h>
#include<string.h>

int main()
{
    int i,j,k,m,n,cl;
    char a[10],b[100],c[100];

    printf("\n ENTER POLYNANOMIAL:");
    scanf("%s",a);

    printf("\n ENTER THE FRAME:");
    scanf("%s",b);

    m=strlen(a);
    n=strlen(b);

    for(i=0;i<m;i++)
    {
        if(a[i]=='1')
        {
            m=m-i;
            break;
        }
    }

    for(k=0;k<m;k++)
        a[k]=a[k+i];

    cl=m+n-1;

    for(i=0;i<n;i++)
        c[i]=b[i];

    for(i=n;i<cl;i++)
        c[i]='0';
```



```
    c[i]='\0';

    for(i=0;i<n;i++)
    {
        if(c[i]=='1')
        {
            for(j=i,k=0;k<m;k++,j++)
            {
                if(a[k]==c[j])
                    c[j]='0';
                else
                    c[j]='1';
            }
        }
    }

    for(i=0;i<n;i++)
        c[i]=b[i];

    printf("\n THE MESSAGE IS: %s",c);

    return 0;
}
```

Output:

```
ENTER POLYNANOMIAL: 1011
ENTER THE FRAME: 10011101
THE MESSAGE IS: 10011101011
```

```
Process exited after 28.25 seconds with return value 0
Press any key to continue ...
```